

Latency Engineering & Tick-to-Trade System Design

Detailed Edition: the Derivation Kept, in Three Parts — Intuition, Tension, Resolution

Robert (Bo) Hou · Robert's Technical Notes · June 2026

Abstract and how to read this. This report keeps the **back-and-forth** of the derivation rather than only the conclusions. It uses a three-part form: **INTUITION** › an initial judgment (often half-right); **TENSION** › where it fails to hold / is challenged; **RESOLUTION** › the corrected conclusion. Part I (latency engineering) is the arsenal for killing tail spikes; Part II (system design) assembles those weapons, with the first three mountains (lock-free, cache, zero-overhead), into a full tick-to-trade link. One sentence runs throughout: **HFT optimizes “determinism” (a bounded worst case), not “average speed”; if it can be contiguous, predictable, and reusable, do not dynamically allocate, do not chase pointers, do not leave a settleable cost to runtime — and the final acceptance test is not a formula, it is replaying the worst historical market data.**

Part I · Latency Engineering: Killing Tail Spikes

1.1 Worldview: Determinism, Not the Average

Why does HFT care about p99.9, not the average? The value is in “can you win / avoid loss in the worst case.” A spike erupts exactly at the **big-move moment (most information, fiercest competition, greatest opportunity)** — which is also when the system is most loaded and most likely to stall; and the cost is **asymmetric**: faster-on-average is a linear small gain, one spike missing/adversely filling is a non-linear large loss. So optimize the bounded worst case and p99.9/p99.99. **Determinism > average speed.** The classic measurement trap is **Coordinated Omission**: if measurement is “send → wait → record → send,” then when the system stalls your sending stalls too, omitting the samples that should have been slow and badly under-estimating the tail; fix it by sending at a **fixed rate**. Tools: HdrHistogram + rdtsc.

1.2 The PAUSE Instruction: Why Busy-Wait Must “Spin Politely”

INTUITION › A busy-wait spins hard, so PAUSE probably tells the CPU “don’t do useless prefetch.”

TENSION › The mechanism is not prefetch. The spin loop `while(flag==0){}` makes the CPU **speculatively issue a pile of loads of flag** into the pipeline; when flag is actually changed by another core, those speculative loads are all stale and must be flushed and redone — the **memory-order violation** penalty. Not “wasted prefetch,” but “wasted speculative loads that get re-punished.”

RESOLUTION › PAUSE tells the CPU “I am spinning,” so it (a) reduces speculative-load pileup and avoids the memory-order-violation flush when flag changes, and (b) yields the physical core’s execution resources to the **hyper-thread sibling**. The key terms are **speculative load + memory-order violation**, not prefetch.

INTUITION › “One core running two threads” isn’t mainstream, and threads can run across cores anyway.

TENSION › **Hyper-threading (SMT) is real and mainstream** — the asymmetry of 12 physical / 16 logical processors is exactly some cores having SMT on (one physical core = 2 logical, sharing execution units). And “threads across cores” is the OS default, but a disaster for HFT: after migration, L1/TLB are cold, a trailing burst of misses.

RESOLUTION › So PAUSE “yielding to the sibling” is meaningful (two threads share one physical core’s ALUs). HFT uses **CPU pinning + core isolation** (`isolcpus`) to forbid migration and own a core; for determinism it may even **disable SMT / leave the sibling idle** — wasting half a core to buy determinism.

1.3 The List of Spike Sources — Each Maps to a Weapon

(1) **Dynamic memory new/malloc**: traverse a free list (doubly-linked) + bookkeeping + lock + occasional syscall, and scattered allocations miss cache later → **preallocated arena/pool**, no new on the hot path. (2) **Page fault**: data not in physical memory, read from disk, ~a million cycles → `mlockall` + **pre-fault**. (3) **Context switch**: on return TLB/cache are cold, a trailing burst of misses → **pinning + isolation**. (4) **NUMA cross-node**: over the CPU interconnect, 1.5-2× higher → **bind thread and data to one node**. Each fix applies a prior mountain’s mechanism.

1.4 Kernel Bypass and “Is Polling Always Faster?”

The kernel stack is slow in four places: **interrupts** (per-packet interrupt → context-switch storm), **syscall + kernel/user transition**, **data copy** (kernel→user buffer, CPU moves + pollutes cache), **generic protocol stack** (checksums/reassembly/routing). Avoided one by one: interrupts → **poll mode**, copy → **zero-copy**, stack → **minimal user-space stack**. Representatives: DPDK / Solarflare onload / `io_uring`.

INTUITION › Since polling replaces interrupts, polling must always be lower latency than interrupts.

TENSION › **Not necessarily.** Polling is fast only **on the condition** that the CPU polls **full-time** (a dedicated core busy-polling, poll interval $\rightarrow 0$). If the CPU is not dedicated and polling is interleaved with other work, the gap between polls holds processing, a packet may wait, and it can be worse than an interrupt that can preempt immediately.

RESOLUTION › HFT meets the condition (willing to burn a core), so it polls — fast and stable. **General rule: every “faster” has a precondition and fails without it** — which is why kernel bypass + busy-poll + core isolation must be deployed **as a set**.

INTUITION › To avoid interrupts, just set a flag: the NIC writes it, the CPU polls it each time, and NIC and CPU are decoupled.

TENSION › That design is essentially DPDK’s **descriptor ring**: actually a **ring (many slots)** not a single flag; the NIC marks a status bit “new packet,” the CPU polls those bits. And that ring **is a lock-free SPSC ring buffer** (NIC produces, CPU consumes).

RESOLUTION › The key synchronization detail is exactly the lock-free mountain: “when status reads ready, the packet data must already be DMA-complete” = **release/acquire memory ordering** (the dual edge from the SPSC report). Here lock-free (ring + ordering) and latency engineering (busy-poll + no interrupts) are stitched together at the NIC — you independently designed the core of a poll-mode driver.

Part I summary. Every item — busy-poll (burn a core), core isolation (dedicate a core), disabling SMT (waste half a core), kernel bypass (dedicate the NIC + write your own stack), locking memory (hold and never return it) — is at root the same trade: **sacrifice resource utilization and generality for worst-case determinism**, the opposite of a throughput system’s direction.

Part II · Tick-to-Trade System Design

2.1 Decomposing “a Million msg/s” into Three Layers

Given one exchange feed, peak **1M msg/s**, each 32–48 bytes binary, strategy time an **exogenous 10µs/message**. “1M/s” must be split: **average vs peak** (design for the peak = biggest opportunity + easiest to stall); **burstiness** (clustered arrivals, variance far above Poisson, hence a buffer is mandatory); **per-message budget** (peak $\Rightarrow \sim 1\mu\text{s}$ each, killing every syscall/malloc/lock/miss-heavy structure, so the first four mountains go from “options” to “necessities”).

2.2 Capacity: Throughput $\neq$ Latency, the Cliff, the Useless Average

INTUITION › A single core does the whole link in $10+1+2 = 13\mu\text{s}$, so one core handles at most 76,923/s.

TENSION › **Conflates latency with throughput**. The link is a **pipeline**: receive, read, strategy are overlapping stages — while strategy handles message 1, the NIC is already receiving 2 and 3. **Throughput is set by the slowest (bottleneck) stage**, not total latency as a divisor.

RESOLUTION › Strategy 10µs is the bottleneck $\Rightarrow$ single-core throughput = $1/10\mu\text{s} = 100\text{k/s}$ (not 76,923). Peak 1M/s $\Rightarrow$ stability needs $c > \lambda/\mu = 10$ cores.

INTUITION › 32 cores, 31% util, queueing gives average wait $0.00104\mu\text{s} \approx 0$ — good enough.

TENSION › **The average is the most useless metric** (as Part I already said, look at the tail). And M/M/c assumes Poisson while real feeds are **bursty**. The “limit the variance” instinct is the right direction — control the tail, not the average.

RESOLUTION › Compute the real **p99**: 16 cores average only 0.096µs but **p99.9 is 6.7µs (100×)**; 32 cores under Poisson have even p99.99 queueing ≈ 0 . **Over-provisioning to 31% util buys not a lower average (already ≈ 0 at 16 cores) but a tail safety margin under bursts.**

Cores	Util	Avg Wq	p99.9 (Poisson)
11	91%	6.82µs	—(barely stable)
16	62%	0.096µs	6.7µs ← the average lies
20	50%	0.0037µs	1.3µs
32	31%	$\approx 0\mu\text{s}$	$\approx 0\mu\text{s}$

The utilization cliff. From 11 cores (91%) to 16 cores (62%), queueing **drops ~70×** — **the closer to 100% utilization, the more non-linearly latency explodes.** This is queueing theory's key intuition and the reason to over-provision: stay far from the cliff so a burst never nears saturation.

2.3 Poisson vs Burst: Can Subdividing Time Model a Burst?

This was the longest and most valuable exchange — repeatedly pressing “can real p99.9 even be computed?”

INTUITION › Feeds are bursty, but at 1 μ s granularity “one arrival at a time” is fine; if not, subdivide to nanoseconds. By a calculus intuition, fine-enough time units let Poisson approximate a burst.

TENSION › **Poisson is memoryless** — arrivals independent, the past does not affect the future. But a burst is **self-exciting / has memory**: one cluster makes the next more likely (arrivals come in groups). Subdividing time only refines granularity; **each small unit is still memoryless Poisson**, and stitched together still lacks cross-segment self-correlation. Memory is a **structural** gap, not a resolution one.

RESOLUTION › Subdivision $\Rightarrow$ a non-homogeneous Poisson process (a deterministic time-varying rate), still assuming “burst timing known + within-unit independence”; reality is the random self-exciting **Hawkes process** (variance far above Poisson). Estimating a burst's tail with Poisson **systematically under-estimates** p99.9.

INTUITION › Then use a Fourier idea: any intensity curve $\lambda(t)$ can be approximated by frequency components; surely someone does this.

TENSION › **You are approximating “one deterministic $\lambda(t)$ curve” (= NHPP).** But the real $\lambda(t)$ is **random (Cox) and self-exciting (Hawkes)** — not a curve you can draw in advance but a random object **generated recursively by its own history**, different every realization. Fourier can fit “one realized curve,” not give “a p99.9 valid for all random paths.”

RESOLUTION › “After the fact, drawing $\lambda(t)$ over the two seconds that happened” can be approximated, but **capacity planning is done in advance**, and tomorrow's clusters land at different times/intensities. Approximating yesterday $\neq$ guaranteeing tomorrow. **So real p99.9 comes only from replaying real historical feeds** (which carry the true cluster structure); queueing theory is only for “how many cores to start with” and “seeing the cliff.”

Resolution of this section. Within the M/M/c model you specified, the microsecond-Poisson computation is unimpeachable and is entirely adequate for choosing core count (≥ 11 stable, 32 for margin) — you are right here, not by accident. The only gap is “model vs reality”: real bursts are self-exciting, and smooth Poisson at any granularity under-estimates the tail. **The two do not conflict: Poisson estimates for sizing, replay for acceptance.**

2.4 Parallelism: SPMC or Sharded SPSC?

INTUITION › For multiple cores, use SPMC — 32 cores share one queue. SPMC itself can be lock-free with tokens, so no problem.

TENSION › SPMC can indeed be lock-free with per-slot tokens (the MPMC-report mechanism). But **lock-free $\neq$ contention-free**: 32 consumers CAS the same head_ $\Rightarrow$ that cache line **ping-pongs** across cores’ L1 $\leftrightarrow$ L3. Lock-free only guarantees “someone progresses,” not “no contention.” And random dispatch **reorders** a symbol’s messages.

RESOLUTION › The instinct “adding write complexity just to have many consumers is odd” is the right direction: **don’t use SPMC, use sharded SPSC** — hash by symbol into 32 **independent queues**, one per core. A single consumer does a plain head_++, **no CAS, no contention**; a symbol always lands on the same core $\Rightarrow$ **ordering for free**. Eliminate contention by architecture (sharding), not by muscling through it with a lock-free queue.

Contention point clarified: the SPMC buffer has many slots (can be $\gg 32$, spread over many cache lines — **not** the contention point); contention is only on the **one** head_ that 32 cores fight over. A bigger buffer does not help; the bottleneck is that single shared head. After sharding, each queue’s head is touched by only one core, staying hot in its L1.

2.5 DDIO: Is the Queue in Memory or in L3?

Another exchange that only closed after several rounds — the crux is separating “logical address” from “physical location.”

INTUITION › Since the NIC uses DDIO to write straight into L3, not memory, why design 32 queues in memory at all? Why not dispatch directly on L3? L3 is shared, so aren’t 32 independent queues just 32 partitions of L3?

TENSION › **A program cannot operate on L3 directly.** A CPU core **can only address by memory address**; cache is transparent and not programmable; each L3 line is bound to a memory address by its **tag**, and you have no “L3 address” to use. So data **must** have a memory address, and the queue **must** be organized by memory addresses — the shared convention between NIC and CPU. L3 is also **not pre-partitioned**; it caches by access.

RESOLUTION › But your insistence that “the data is still read from L3” is **entirely correct** — there is no real disagreement. **“Queue in memory” is the address (its registry); “data read from L3” is the physical location** — two facets of one datum. `load [addr]` reads by memory address and hits in L3, one action. DDIO saves the “to DRAM then read back” round-trip, not the data’s memory address.

INTUITION › Then those 32 independent queues, mapped back onto L3, should also be independent, right?

TENSION › Right in direction, but “logically independent” does not **automatically** mean “cache independent.” If two queues’ head/tail sit too close in memory, on the **same cache line**, they false-share and ping-pong even though logically independent (mountain II).

RESOLUTION › The prerequisite is aligning each hot variable to its own line with `alignas(64)`. Sharding removes the “many cores fight one head” contention; you still need `alignas` to prevent “different queues’ heads crammed on one line” false sharing — both, for true cache independence.

2.6 Buffer Sizing and Degradation Under Burst Overrun

INTUITION › With 32 cores the average wait ≈ 0 , so in theory the buffer needn’t be large; even one cache line per core is enough.

TENSION › Do not conflate two scales: the **cache line (64 bytes)** is the false-sharing alignment unit (head/tail each take one); **buffer size** is the burst-buffering unit, set by “how many messages to buffer.” One line holds 16 floats / one 64-byte message — nowhere near enough for a burst.

RESOLUTION › The theoretical buffer (by average) ≈ 0 , but **in practice it must be tens of times larger to prevent burst overrun** — sized to the worst burst’s instantaneous backlog; yet not blindly large (beyond L2 it slows, a deep queue has high latency). The compromise: **large enough for the worst historical burst, still fitting in L2**, set by replaying history and watching the peak backlog.

INTUITION › No design stops a black-swan overrun, so when the queue is full, drop old not new — that’s an LRU cache.

TENSION › Drop old not new is right (market data **expires**; acting on a stale quote is adverse selection — a loss; **freshness is value**). But it is **not LRU**: LRU evicts by “access history” for hit rate; here we evict by “arrival time / freshness.” A better name is stale-data eviction.

RESOLUTION › The best form is **conflation**: a new quote for a symbol **overwrites** that symbol's stale quote in the queue, so a burst of updates for one symbol collapses to its latest and the queue does not blow up. Conflation keys on symbol, logically a hash; but symbol ids are usually **dense small integers**, so index an **array directly** (latest[symbol]) rather than `std::unordered_map` — $O(1)$, no collisions, no malloc, cache-friendly.

2.7 Why “Same Diagram” Is Still Hard

INTUITION › Deriving all this, system design didn't feel very hard — unlike a math proof you puzzle over. Is it just “ordinary big-data system + queueing + real-time”?

TENSION › It goes smoothly because the **thinking type differs and you came prepared**: a math proof is **convergent** (force out a unique, counter-intuitive construction — hence you get stuck); system design is **divergent / trade-off** (judge among known options — hence it flows). And the bricks (lock-free, cache, zero-overhead) were fired in the first four mountains; here you only assemble.

RESOLUTION › The **skeleton** is indeed shared with big-data, but **microsecond-scale + deterministic** forces each component into a wholly different implementation: the same “queue” is Kafka (ms/disk/net) in big-data, lock-free SPSC (ns/cache/single-core) in HFT — six orders apart. **Many can draw the diagram; few can make each component work deterministically at nanosecond scale** — that is precisely the first four mountains.

Component	Ordinary big-data	HFT
Latency goal	ms-s, avg/throughput	µs-ns, worst/determinism
Queue	Kafka (disk/net/ms)	lock-free SPSC ring (ns)
Sharding	across machines (network)	across cores (cache line)
Memory	GC, tens-of-ms pauses OK	C++ preallocated, no GC/malloc/fault
Network	kernel TCP stack	kernel bypass
Fault tol.	replicas/retries/eventual	almost no retry (retry=latency=miss)

Closing. System design “looks easy” because it **stands on the shoulders of the first four mountains**: the bricks are already fired and verified; here we only assemble. All five converge to one sentence: **if it can be contiguous, predictable, and reusable, do not dynamically allocate, do not chase pointers, do not leave a settleable cost to runtime; and for the final acceptance test, replay the worst historical market data.**